Grupo: Balbuena Juan, Sangalli Luciano, Romero Gonzalo, Romero Valentin, Zaragoza Nahuel

1. Requerimientos y Casos de Prueba (Collatz)

Requerimientos Funcionales (R):

- **R_1:** Solicitar y aceptar un número entero positivo mediante entrada por teclado.
- **R_2:** Comprobar la conjetura calculando iterativamente (dividiendo por 2 si es par, o multiplicando por 3 y sumando 1 si es impar) hasta alcanzar el valor 1.
- **R_3:** Imprimir en pantalla el número de partida original y la cantidad total de iteraciones necesarias.

Requerimientos No Funcionales (F):

- **F_1:** El límite máximo del número entero a validar debe estar restringido a 1999.

Matriz de Trazabilidad (RTMX)

ID Prueba	Tipo de Prueba	R_1	R_2	R_3	F_1
T_1 (Entrada: 20)	Caja Negra (Funcional)	X	X	X	
T_2 (Entrada: 2000)	Caja Negra (Funcional)	X			X
T_3 (Entrada: 1)	Caja Blanca (Unitario)	X	X		

2. Inspección de Código y Densidad de Defectos

Durante la **Sesión 1** de inspección estática del código provisto, se detectaron los siguientes defectos:

1. **Semántico/Sintáctico:** Las operaciones `num // 2` y `3*num + 1` no reasignan valor a la variable (debería ser `num = num // 2`).
2. **Semántico:** La función retorna `iter`, una variable inexistente (debería ser `return iteraciones`).
3. **Sintáctico:** Error de formato en el print final (falta %d), uso de variable indefinida `j`, y falta cerrar el paréntesis.

Cálculo de Densidad Inicial:
Defectos detectados: 3. Tamaño del programa (S): 18 Locs.
Densidad (δ_0) = 3 / 18 = **0.166 defectos/Loc**.

3. Desempeño Organizacional y Modelo de Garantía

Se cuenta con una organización con una efectividad de detección (PCE) del 89% (0.89), una tasa de defectos liberados (δ_r) de 0.12 def/FP y un proyecto de S = 100 FP.

a. Expectativa de defectos totales (μ_0):

Sabemos que los defectos liberados equivalen a: 0.12 def/FP × 100 FP = 12 defectos.
La efectividad PCE se define como los defectos detectados sobre el total, por lo que los defectos liberados son el porcentaje complementario (11%).
$0.11 \times \mu_0 = 12 \rightarrow$ **μ_0 = 109.09 defectos totales esperados.**

b. Densidad de defectos al finalizar construcción (δ_0):

$\delta_0 = \mu_0 / S = 109 / 100 =$ **1.09 defectos/FP.**

c. Defectos a detectar durante V&V:

Detectados = μ_0 − Liberados = 109 − 12 = **97 defectos.**

d. Comportamiento frente a la detección temprana (60 defectos en 3 días):

Si en los primeros 3 días se han detectado 60 defectos (aproximadamente el 62% del total proyectado para toda la fase), indica una curva de inyección de defectos atípicamente alta, o una rigurosidad inicial extrema. Es estadísticamente probable que el total final real (μ_0) supere considerablemente la línea base de 109 defectos.

4. Conceptos Teóricos Adicionales

Parámetro TEP (Tiempo Esperado de Prueba): En un modelo Kanban para calidad, el TEP indica el umbral de tiempo estadístico necesario para ejecutar las validaciones y alcanzar un nivel de confiabilidad aceptable. Su criticidad radica en que forzar la liberación antes de cumplir el TEP incrementa exponencialmente el riesgo de liberar fallas críticas al cliente.

Impacto de Multas en el Equilibrio: En el modelo de garantía, la aplicación de multas encarece fuertemente el costo de los defectos post-liberación. Esto desplaza el punto de equilibrio económico del proyecto, forzando a la organización a invertir más tiempo y recursos en las fases de V&V internas para minimizar los defectos residuales, ya que el costo de la calidad es ahora inferior al costo de la multa.

5. Herramienta Python y CI/CD (json2csv.py)

El código debe ser implementado y gestionado a través de GitHub utilizando un framework de integración continua. El siguiente script implementa la estrategia de filtros de forma robusta.

```
      import sys import json import csv import argparse def
convert_json_to_csv(input_file, output_file, delimiter): try: data =
json.load(input_file) if not data or not isinstance(data, list): raise ValueError("JSON
debe contener una lista.") writer = csv.DictWriter( output_file,
fieldnames=data[0].keys(), delimiter=delimiter ) writer.writeheader()
writer.writerows(data) except Exception as e: print(f"Error procesando archivo: {e}",
file=sys.stderr) sys.exit(1) def main(): parser = argparse.ArgumentParser()
parser.add_argument("-i", "--input", help="Entrada (default stdin)")
parser.add_argument("-o", "--output", help="Salida (default stdout)")
parser.add_argument("-d", "--delimiter", default=",") args = parser.parse_args() in_st
= open(args.input, 'r', encoding='utf-8') if args.input else sys.stdin out_st =
open(args.output, 'w', encoding='utf-8', newline='') if args.output else sys.stdout
try: convert_json_to_csv(in_st, out_st, args.delimiter) finally: if args.input:
in_st.close() if args.output: out_st.close() if __name__ == "__main__": main()
```

Flujo CI/CD Requerido en GitHub Actions

Se deberá crear un pipeline que ejecute secuencialmente los siguientes controles de calidad obligatorios sobre cada Pull Request:

- **Formateo y Estilo:** Validación con `ruff`, `black` y convenciones de `PEP8 / PEP257`.
- **Análisis Estático de Tipos:** Verificaciones utilizando `mypy` y `pyright`.
- **Seguridad:** Escaneo pasivo con `bandit` (asegurando la exclusión de Trufflehog).
- **Testing:** Ejecución de `pytest` asegurando una cobertura de código (coverage) superior al 85%